

Nachbildung mathematischer Operationen durch ML Modelle

Seminararbeit

an der Fakultät Wirtschaft
im Studiengang Data Science & künstliche Intelligenz

an der
DHBW Ravensburg

Autor: Elias Pfeiffer
Modul: Seminar
Dozent: Martin Zeafferer
Abgabefrist: 16.03.2025

Inhaltsverzeichnis

Abkürzungsverzeichnis	III
Abbildungsverzeichnis	IV
Formelverzeichnis	V
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung & Vorgehensweise	1
2 Theoretische Grundlagen	3
2.1 Lernbarkeit mathematischer Operationen	3
2.2 Entscheidungsbäume & Random Forests	3
2.3 Support Vector Machines	4
2.4 Neuronale Netze	5
2.5 Evaluationsmetriken	5
2.6 Generalisierung & Extrapolation	5
3 Methodik	6
3.1 CRISP-DM	6
3.2 E	7
4 Versuchsdurchführung	8
4.1 Konstruktion der Datensätze	8
4.2 Training der Modelle	9
4.2.1 Support Vector Regression	9
4.2.2 Entscheidungsbäume & Random Forest	9
4.2.3 Neuronale Netze	10
5 Evaluation der Experimentergebnisse	12

5.1	Support Vector Regression	12
5.2	Entscheidungsbäume & Random Forest	12
5.3	Neuronale Netze	12
6	Diskussion	13
6.1	Diskussion der Ergebnisse & Limitationen	13
6.2	Mögliche Weiterentwicklungen & Optimierungen	13
7	Fazit	14
	AnhangA: O'REILLY: Bootstrap	15
	Quellen- und Literaturverzeichnis	17

Abkürzungsverzeichnis

DHBW Duale Hochschule Baden-Württemberg

CRISP-DM Cross Industry Standard Process for Data Mining

SVM Support Vector Machine

SVR Support Vector Regression

RBF Radial Basis Function

ML Machine Learning

CSV Comma-separated values

Bagging Bootstrap Aggregation

ReLU Rectified Linear Unit

Abbildungsverzeichnis

Formelverzeichnis

$y = x_1 + x_2$	4.1
$y = x_1 * x_2$	4.2

1 Einleitung

1.1 Motivation

„Mathematische Argumentation kann schematisch als die Anwendung der Kombination von zwei Fähigkeiten betrachtet werden, die wir Intuition und Einfallsreichtum nennen können.“

- Alan Turing

Mathematisches Denken basiert nach Alan Turing nicht allein auf der mechanischen Ausführung formaler Regeln, sondern auf der Grundlage von zwei Fähigkeiten. Zum einen die Intuition, worunter das Erkennen und Verallgemeinern von Mustern verstanden werden kann und zum anderen der Einfallsreichtum, welcher das strukturierte Anwenden von Regeln zur Lösung mathematischer Probleme bezeichnet. Diese Unterscheidung ist bis heute von zentraler Bedeutung für das Verständnis künstlicher Intelligenz und gewinnt im Kontext moderner Machine Learning (ML)-Methoden an Relevanz.

Aktuelle ML-Modelle zeichnen sich durch die Fähigkeit der Mustererkennung aus. Sie sind in der Lage, Zusammenhänge in Daten zu erkennen, ohne dass die zugrunde liegenden Regeln vorgegeben werden müssen. Diese Herangehensweise entspricht in Turings Zitat vor allem der Intuition. Das Gegenstück bildet die klassische Mathematik, bei der mathematische Operationen wie Addition, Multiplikation oder Division durch klar definierte, deterministische Verfahren durchgeführt werden und damit Turings Einfallsreichtums entspricht.

Die zunehmende Leistungsfähigkeit von ML-Modellen wirft daher die grundlegende Frage auf, inwieweit solche Modelle nicht nur mathematische Zusammenhänge approximieren, sondern tatsächlich in der Lage sind, mathematische Operationen strukturell zu erlernen und diese auf bisher unbekannten Wertebereichen anzuwenden.

1.2 Problemstellung & Vorgehensweise

Vor diesem Hintergrund untersucht die vorliegende Arbeit die Fähigkeit verschiedener ML-Modelle, grundlegende mathematische Operationen nachzubilden. Während viele Modelle im Bereich der Trainingsdaten geringe Vorhersagefehler erzielen, bleibt häufig unklar, ob sie die mathematische Struktur erfasst haben oder lediglich das Ergebnis approximieren. Diese Untersuchung ist insbesondere dann von hoher Bedeutung, wenn Modelle auf Inputs außerhalb des Wertebereichs des Trainings angewendet werden sollen, da in diesem Wertebereich mangelnde Generalisierungsfähigkeiten der Modelle sichtbar gemacht werden.

Ziel dieser Arbeit ist es daher, systematisch zu analysieren, welche Modellklassen grundlegende mathematische Operationen, wie Addition oder Multiplikation nachbilden können und inwieweit eine Generalisierung auf bisher unbekannten Wertebereichen gelingt. Daher liegt der Fokus weniger auf maximaler Vorhersagequalität innerhalb des Trainingsbereichs, sondern auf der Untersuchung der Grenzen und Potenziale der eingesetzten Modelle.

Methodisch orientiert sich die Arbeit an ausgewählten Phasen des Cross Industry Standard Process for Data Mining (CRISP-DM)-Prozessmodells. Relevant sind hierbei die Phasen *Business Understanding*, in der die Zielsetzung der Untersuchung definiert wird, *Data Understanding* und *Data Preparation*, welche die Konstruktion geeigneter Datensätze für mathematische Operationen umfassen, sowie *Modeling* und *Evaluation*. In diesen Phasen werden zum einen die unterschiedlichen ML-Modelle trainiert, zum anderen werden sie hinsichtlich ihrer Prognosegüte und ihrer Generalisierungsfähigkeiten miteinander verglichen. Wirth und Hipp, 2000 und Shearer, 2000 liefern eine detaillierte Beschreibung des gesamten CRISP-DM-Modells.

Durch dieses strukturierte Vorgehen soll ein nachvollziehbarer, experimenteller Rahmen geschaffen werden, der die Bewertung der Potenziale und Limitationen datengetriebener Modelle bei der Nachbildung mathematischer Operationen ermöglicht, insbesondere vor dem Hintergrund, dass es nicht immer eindeutig ist, warum bestimmte ML-Modelle bei bestimmten Datensätzen bessere oder schlechtere Ergebnisse liefern.

2 Theoretische Grundlagen

2.1 Lernbarkeit mathematischer Operationen

2.2 Entscheidungsbäume & Random Forests

Entscheidungsbäume gehören zu den überwachten Lernverfahren, die sowohl für Klassifikations- als auch Regressionsaufgaben verwendet werden können. Sie modellieren Entscheidungsprozesse in Form einer Baumstruktur, wobei zwischen inneren Knoten und Blattknoten unterschieden wird. Innere Knoten repräsentieren ein Merkmal oder Attribut der Eingabedaten, während Blattknoten die Zielklasse (bei Regressionsbäumen handelt es sich hierbei um einen numerischen Wert) enthalten. Die Regression bzw. Klassifikation beginnt am Wurzelknoten, an dem ein Feature des betrachteten Datenpunkts betrachtet wird. Abhängig vom Ergebnis der hier getroffenen Entscheidung des Baumes, wird der Datenpunkt entlang der Kante einem Kindknoten zugewiesen. Dieser Prozess wird rekursiv fortgeführt, bis ein Blattknoten erreicht ist, dessen Wert die Modellvorhersage darstellt (vgl. Li, 2024, S. 77).

Für Regressionsaufgaben werden Regressionsbäume verwendet, bei denen jedem Blatt ein konstanter Vorhersagewert zugeordnet ist. Dadurch sind Entscheidungsbäume in der Lage nichtlineare Zusammenhänge abzubilden. Sie neigen dazu, die Trainingsdaten sehr exakt zu approximieren, was insbesondere bei tiefen Bäumen zu Overfitting und eingeschränkter Generalisierung führen kann. Die Komplexität eines Baumes wird daher oft durch die Begrenzung der Baumtiefe kontrolliert.

Random Forests stellen eine Ensemble-Methode dar, die auf Bootstrap Aggregation (Bagging) aufbaut. Dabei werden mehrere Entscheidungsbäume unabhängig voneinander auf zufälligen Stichproben der Trainingsdaten trainiert (Ziehen mit Zurücklegen). Auch wird in jedem Split nur eine Teilmenge aller Features betrachtet. Die Finale Vorhersage ergibt sich aus der Kombination der Ergebnisse aller Einzelmodelle. Bei Regressionsaufgaben kommt in diesem Schritt häufig der Mittelwert zum Einsatz. Durch die Kombination vieler unabhängiger Modelle wird durch Bagging die Varianz von Vorhersagen reduziert, ohne die Fähigkeit zur Modellierung nichtlinearer Zusammenhänge einzuschränken.

Im Kontext dieser Arbeit sind Entscheidungsbäume und Random Forests besonders relevant, da sie anders als andere Modelle den Eingaberaum stückweise zerlegen. Dies führt innerhalb des Trainingsbereichs zu präzisen Vorhersagen, kann jedoch negative Auswirkungen auf die Generalisierungsfähigkeit eines Modells haben.

2.3 Support Vector Machines

Support Vector Machine (SVM) sind überwachte Lernverfahren, die ursprünglich für Klassifikationsprobleme entwickelt wurden, sich jedoch auch für Regressionsaufgaben abwandeln lassen. Zentrales Ziel einer SVM ist es, eine Entscheidungsfunktion zu bestimmen, die eine möglichst gute Generalisierung auf unbekannten Daten erlaubt. Dieses Ziel wird durch das Prinzip der Margin-Maximierung (Der Minimale Abstand zur trennenden Hyperebene ist maximal) erreicht, wodurch die Anpassung an die Trainingsdaten und Komplexität des Modells kontrolliert bzw. angepasst wird. (vgl. Li, 2024, 127ff).

Die Lösung einer SVM basiert ausschließlich auf den sogenannten Support Vektoren, bei denen es sich um die Datenpunkte handelt, welche die Entscheidungsgrenze für das Modell bilden. Diese Punkte haben den größten Einfluss auf die Bildung des Modells und der Regressionsfunktion (vgl. MathWorks, n. d.).

Für Regressionsprobleme wird die Support Vector Regression (SVR) eingesetzt. Ziel der SVR ist es, eine Funktion zu erlernen, die Zielwerte möglichst genau approximiert und gleichzeitig die Komplexität gering zu halten. Ein zentrales Element ist hierbei die ϵ -Abweichung. Abweichung zwischen vorhergesagtem und tatsächlichen Wert werden nur dann bestraft, wenn sie den definierten Toleranzbereich ϵ überschreiten. Kleine Fehler innerhalb dieses Bereiches bleiben daher unberücksichtigt. Dadurch wird das Modell robuster gegenüber Rauschen und Schwankungen in den Trainingsdaten (vgl. Ma & Guo, 2014, S. 271).

Die Balance zwischen Modellkomplexität und Anpassung an die Trainingsdaten wird durch den Parameter C gesteuert. Ein hoher Wert für C führt zur stärkeren Bestrafung von Abweichungen, wodurch sich das Modell stärker an die Trainingsdaten anpasst. Kleinere Werte begünstigen glattere Modelle und somit die Generalisierungsfähigkeit des Modells auf unbekannten Daten. Für die Nachbildung mathematischer Operationen ist diese Eigenschaft relevant, da hier nicht nur exakte Werte innerhalb des Trainingsbereichs relevant sind, sondern auch Vorhersagen für unbekannte Inputs abgedeckt werden sollen (vgl. Ma & Guo, 2014, S. 272).

Viele Zusammenhänge, z.B. bei nichtlinearen mathematischen Operationen, lassen sich nicht durch lineare Modelle beschreiben. SVMs begegnen diesem Problem durch den Einsatz von Kernelmethoden. Beim sogenannten Kernel-Trick wird ein lineares Modell in einem höherdimensionalen Feature-Raum angewandt. Es wird durch den Kernel Trick versucht, ein lineares Modell auf ein nichtlineares Klassifizierungsproblem anzuwenden (vgl. Li, 2024, S. 153).

In der Praxis werden allerdings verschiedene Kernelfunktionen eingesetzt. Dazu zählen der lineare Kernel, der Polynomkernel sowie der Radial Basis Function (RBF)-Kernel. Insbesondere der RBF-Kernel erlaubt die Modellierung nichtlinearer Zusammenhänge und

spielt daher eine zentrale Rolle bei der Approximation mathematischer Operationen. Im Kontext der Arbeit dienen Support Vector Machines als Referenzmodell zur Untersuchung der Forschungsfrage.

2.4 Neuronale Netze

2.5 Evaluationsmetriken

2.6 Generalisierung & Extrapolation

3 Methodik

3.1 CRISP-DM

Das CRISP-DM-Modell stellt einen Prozess für Data-Mining-Projekte dar. Es wurde 1996 von einem Zusammenschluss von Unternehmen wie Daimler Chrysler, SPSS, NCR und OHRA entwickelt, um eine einheitliche Vorgehensweise in Data-Mining-Projekten zu schaffen. Ziel des CRISP-DM ist es, zum einen die Effizienz in Projekten zu steigern. Zum anderen soll Wiederholbarkeit und Vergleichbarkeit erhöht werden (vgl. Shearer, 2000), (vgl. Wirth & Hipp, 2000).

Das Modell beschreibt den Lebenszyklus eines Data-Mining-Projekts und gliedert sich in 6 Phasen, welche einen iterativen Kreislauf bilden. Der zyklische Ablauf eines auf CRISP-DM basierenden Projektes ermöglicht die Einbeziehung gewonnener Erkenntnisse in zukünftige Projektphasen und Zyklen (vgl. Shearer, 2000). Die sechs zyklischen Phasen des CRISP-DM sehen folgendermaßen aus:

1. **Business Understanding:** In der Anfangsphase werden die Projektziele und Anforderungen definiert. Ziel dieser Phase ist es ein vorhandenes Problem in eine Data-Mining-Aufgabe zu überführen (vgl. Wirth & Hipp, 2000).
2. **Data Understanding:** Diese Phase umfasst das Sammeln und Beschreiben der Daten. Ziel ist es, einen Überblick über Qualität und Zusammenhänge in den Daten zu erhalten (vgl. Shearer, 2000).
3. **Data Preparation:** Bei Data Preparation werden die Daten für die Analyse vorbereitet. Zu den Aufgaben zählen die Datenauswahl, deren Bereinigung und eventuelle Transformation. Die Qualität der Modelle hängt von der Sorgfalt dieser Phase ab (vgl. Shearer, 2000).
4. **Modeling:** In dieser Phase werden die vorhandenen Daten für Modellierungsverfahren genutzt. Mehrere Modelle können hierbei parallel erstellt und anhand definierter Kriterien bewertet werden (vgl. Shearer, 2000).
5. **Evaluation:** Das erstellte Modell wird in dieser Phase in Bezug auf die festgelegten Geschäftsziele kritisch bewertet. Die Evaluation dient der Sicherstellung der Qualität und zeigt auf, ob in weiteren Iterationen Verbesserungen notwendig sind (vgl. Shearer, 2000).
6. **Deployment:** In der Abschlussphase des Modells werden die Erkenntnisse oder Modelle in die Praxis überführt. In dieser Phase kann ein Ergebnisbericht oder eine vollständige Integration eines Modells in ein System erfolgen.

3.2 E

4 Versuchsdurchführung

Dieses Kapitel umfasst die praktische Umsetzung des Experiments zur Nachbildung mathematischer Operatoren durch ML-Modelle. Der Code wurde vollständig in Python verfasst und kann dem zugehörigen GitHub-Repository entnommen werden.

4.1 Konstruktion der Datensätze

Zur Untersuchung der Fähigkeit von ML-Modellen, grundlegende mathematische Operationen nachzubilden, wurden synthetische Datensätze generiert. Die Verwendung künstlich erzeugter Daten ermöglicht eine vollständige Kontrolle über die verwendete mathematische Operation und vermeidet Störungen, die in realen Daten auftauchen können. Dadurch kann gezielt analysiert werden, ob ein Modell die mathematische Beziehung zwischen Eingabe- und Zielvariablen erlernt oder lediglich eine Approximation im Trainingsbereich vornimmt.

Für alle betrachteten Rechenoperationen wurde derselbe Grundaufbau verwendet. Die Inputdaten bestehen aus zwei kontinuierlichen Variablen x_1 und x_2 , die unabhängig voneinander aus einer Gleichverteilung im Intervall $[-10, 10]$ stammen. Die Stichprobe umfasst $N = 100000$ Datenpunkte, um eine ausreichende Abdeckung des Eingaberaums sicherzustellen und gute Trainingsbedingungen zu schaffen.

Die Zielvariable y wurde deterministisch aus den Eingabevariablen berechnet. Für den Additionsdatensatz gilt dabei

$$y = x_1 + x_2 \tag{4.1}$$

während für den Multiplikationsdatensatz die Formel

$$y = x_1 * x_2 \tag{4.2}$$

verwendet wurde.

Datensätze für weitere Operationen und in anderen Intervallen können nach demselben Prinzip generiert werden. Da keine zufälligen Störgrößen hinzugefügt wurden, enthalten die Datensätze kein Rauschen, wodurch das Erlernen mathematischer Operationen der trainierten Modelle isoliert untersucht werden kann.

Die erzeugten Daten wurden in tabellarischer Form mit den Spalten x_1 , x_2 und y gespeichert und als Comma-separated values (CSV)-Dateien exportiert. Diese Struktur erlaubt eine direkte Verwendung in den Trainings- und Testschritten. Gleichzeitig wird sichergestellt, dass Unterschiede in den Ergebnissen auf die jeweilige Modellarchitektur und Lernprozess zurückzuführen sind und nicht auf Unterschiede in den Daten.

Durch diese kontrollierte Konstruktion der Datensätze wird eine experimentelle Grundlage geschaffen, um die Lern- und Generalisierungsfähigkeiten der eingesetzten ML-Modelle systematisch zu analysieren.

4.2 Training der Modelle

4.2.1 Support Vector Regression

Im Rahmen dieser Arbeit wurden zwei Modelle auf Basis der Support Vector Regression trainiert, um dessen Eignung zur Nachbildung mathematischer Operationen zu untersuchen. Für die Operatoren *Addition* und *Multiplikation* wurden jeweils zwei SVR-Modelle mit unterschiedlichen Kernel-Funktionen trainiert.

Vor dem Training wurden die Eingabedaten mit dem *StandardScaler* standardisiert. Diese Verarbeitung ist für SVR notwendig, da die Regressionsfunktion des Modells durch Abstände im Feature-Raum optimiert wird und somit unterschiedliche Skalierungen einen negativen Einfluss auf das Gesamtmodell besitzen. Die beiden Modelle wurden auf einem Trainingsdatensatz für jeweils einen Operator trainiert, der in Form eines 80/20-Splits von den in Kapitel 4.1 generierten Datensätzen abgetrennt wurde. Anschließend wurde das Modell auf dem zugehörigen Testdatensatz und dem separat generierten Datensatz zur Generalisierung.

Für die Addition wurde eine SVR mit linearem Kernel eingesetzt. Der Regulierungsparameter $C = 10$ legt fest, wie stark Abweichungen zwischen Vorhersage und tatsächlichem Wert bestraft werden. Ein hoher Wert begünstigt eine präzise Anpassung an die Trainingsdaten. Der Parameter $\epsilon = 0,01$ definiert einen schmalen Toleranzbereich, wodurch eine präzise Annäherung der Zielvariable angestrebt wird.

Für die Multiplikation wurde eine SVR mit RBF-Kernel verwendet. Die Parameter C und ϵ wurden identisch gesetzt, um Vergleichbarkeit mit dem linearen Modell sicherzustellen. Der Parameter γ wurde auf *scale* gesetzt, wodurch dessen Wert automatisch bestimmt wird. Dies erlaubt eine adaptive Anpassung an die Trainingsdaten.

Durch die gezielt identische Wahl der Parameter beider Modelle wird der Einfluss der Wahl des Kernels isoliert betrachtet. Die Bewertung der Modellgüte sowie des Generalisierungsverhaltens erfolgt in den folgenden Kapitel anhand definierter Metriken.

4.2.2 Entscheidungsbäume & Random Forest

Für die vier Modelle Entscheidungsbaum (Addition & Multiplikation) und Random Forest (Addition & Multiplikation) wurde ein identischer Versuchsablauf verwendet.

Zunächst wurde der jeweilige Datensatz mittels des 80/20-Trainings-Test-Splits unter-

teilt. Die Modelle wurden ausschließlich auf dem Trainingsteil trainiert und anschließend auf dem Testdatensatz evaluiert. Zusätzlich wurde jedes Modell auf dem separaten Generalisierungsdatensatz angewendet, um das Verhalten außerhalb der Trainingsdaten zu analysieren.

Für die Entscheidungsbäume kamen Regressionsbäume mit begrenzter Baumtiefe zum Einsatz. Die Begrenzung der Tiefe eines Baumes dient der Kontrolle der Modellkomplexität und reduziert die Gefahr von Overfitting. Für jeden Rechenoperator wurde hier ein eigenes Modell trainiert und evaluiert.

Die Random Forest Modelle folgen einem identischen Ablauf, basieren jedoch auf einem Ensemble Ansatz. Dabei werden durch Bagging mehrere Entscheidungsbäume parallel auf zufälligen Stichproben der Trainingsdaten trainiert. Die finale Vorhersage ergibt sich aus dem Durchschnitt der Einzelpredictionen, was zu stabileren Ergebnissen führt. Das Ensemble Modell umfasst 200 einzelne Bäume mit einer maximalen Tiefe von 10.

Durch die einheitlichen Trainings- und Evaluationsprozess ist eine Vergleichbarkeit der Modelle hinsichtlich ihres Lernverhaltens für unterschiedliche Rechenoperatoren gewährleistet.

4.2.3 Neuronale Netze

Für Neuronale Netze wurden im Rahmen der Arbeit ebenfalls zwei separate Feedforward-Netze trainiert. Wie für die vorangegangenen Modelle, bildet ein Neuronales Netz Addition und das andere Multiplikation ab.

Die Eingabedaten bestehen aus den kontinuierlichen Variablen x_1 und x_2 , die vor dem Training standardisiert wurden. Die Verwendung einer Skalierung dient der Stabilisierung des Trainingsprozesses sicher. Die Datensätze wurden im Anschluss durch einen 80/20-Split in Trainings- und Testdaten aufgeteilt.

Beide neuronalen Netze besitzen einen identischen Aufbau, um einen direkten Vergleich zwischen den Rechenoperatoren zu gewährleisten. Die Netze bestehen aus zwei Hidden Layers mit jeweils 32 Neuronen und nutzen die ReLU-Aktivierungsfunktion. Die Ausgabeschicht besteht aus einem einzelnen Neuron ohne Aktivierungsfunktion, wodurch das Modell für Regressionsaufgaben geeignet ist. Durch diesen Aufbau approximiert das Modell sowohl lineare, als auch nichtlineare Zusammenhänge.

Das Training erfolgte mithilfe des Adam-Optimierers und der Mean-Squared-Error-Lossfunktion. Nach 50 Trainingsiterationen wurden die Modelle zunächst auf den Testdatensätzen evaluiert. Zusätzlich wurde für beide Modelle ein zusätzlicher Generalisierungstest durchgeführt, um das Verhalten der Netze auf bislang ungesehenen Wertebereichen zu analysieren.

Durch die Verwendung identischer Architekturen und Trainingsparameter für Addition und Multiplikation wird sichergestellt, dass Unterschiede in den Ergebnissen auf die je-

weilige Datengrundlage zurückzuführen sind.

5 Evaluation der Experimentergebnisse

5.1 Support Vector Regression

5.2 Entscheidungsbäume & Random Forest

5.3 Neuronale Netze

6 Diskussion

6.1 Diskussion der Ergebnisse & Limitationen

6.2 Mögliche Weiterentwicklungen & Optimierungen

7 Fazit

Ich bin ein Fazit.

AnhangA: O'REILLY: Bootstrap
Responsive Web Development



O'REILLY®

Jake Spurlock
Foreword by Dave Winer

Want to read more?

You can [buy this book](#) at **oreilly.com**
in print and ebook format.

Buy 2 books, get the 3rd FREE!

Use discount code: OPC10

All orders over \$29.95 qualify for **free shipping** within the US.

It's also available at your favorite book retailer,
including the iBookstore, the [Android Marketplace](#),
and [Amazon.com](#).



O'REILLY®

Spreading the knowledge of innovators

[oreilly.com](#)

Quellen- und Literaturverzeichnis

- Li, H. (2024). *Machine Learning Methods*. Springer Nature Singapore. <https://doi.org/10.1007/978-981-99-3917-6>
- Ma, Y., & Guo, G. (Hrsg.). (2014). *Support Vector Machines Applications*. Springer International Publishing. <https://doi.org/10.1007/978-3-319-02300-7>
- MathWorks. (n.d.). Was ist eine Support Vector Machine? Verfügbar 18. Januar 2026 unter <https://de.mathworks.com/discovery/support-vector-machine.html>
- Shearer, C. (2000). The CRISP-DM Model: The New Blueprint for Data Mining. *Journal of Data Warehousing*, 5(4), 13–22. <https://mineracaodedados.files.wordpress.com/2012/04/the-crisp-dm-model-the-new-blueprint-for-data-mining-shearer-colin.pdf>
- Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a Standard Process Model for Data Mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*. <https://cs.unibo.it/~danilo.montesi/CBD/Beatriz/10.1.1.198.5133.pdf>

Selbstständigkeitserklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Hirrlingen, 26. Januar 2026

Ort, Datum

Unterschrift

Erklärung zum Einsatz von KI-basierten Werkzeugen

Zur Verwendung KI-gestützter Werkzeuge erkläre ich in Kenntnis des Hinweisblatts „Hinweise zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung wissenschaftlicher Arbeiten, u.a. im prüfungsrechtlichen Kontext“ Folgendes:

- Ich habe mich aktiv über die Leistungsfähigkeit und Beschränkungen der in meiner Arbeit eingesetzten KI-Werkzeuge informiert.
- Bei der Anfertigung der Arbeit habe ich durchgehend eigenständig und beim Einsatz KI-gestützter Werkzeuge maßgeblich steuernd gearbeitet.
- Insbesondere habe ich die Inhalte entweder aus wissenschaftlichen oder anderen zugelassenen Quellen entnommen und diese gekennzeichnet oder diese unter Anwendung wissenschaftlicher Methoden selbst entwickelt.
- Mir ist bewusst, dass ich als Autor/in der Arbeit die volle Verantwortung für die in ihr gemachten Angaben und Aussagen trage.
- Soweit ich KI-gestützte Werkzeuge zur Erstellung der Arbeit eingesetzt habe, sind diese jeweils mit dem Produktnamen, den formulierten Eingaben (Prompts), der Einsatzform sowie der entsprechenden Seiten-/Bereichsreferenzierung auf die Arbeit im KI-Verzeichnis am Ende der Arbeit vollständig ausgewiesen und im Text belegt (z.B. als Fußnote).

KI-Verzeichnis:

KI-gestütztes Werkzeug	Prompts	Einsatzform	Seiten- /Bereichsangabe
ChatGPT	Schreibe eine wissenschaftliche Arbeit mit ca. 25 Seiten mit dem Titel X	Verfassen meines Textes	alles